

CAP776

Programming in Python



Lecture #11

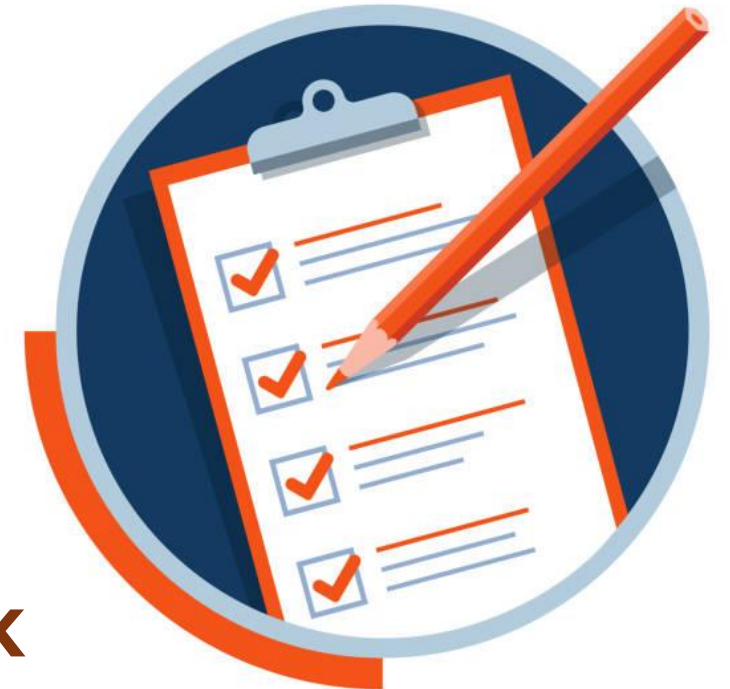
Minor Project #1 debriefing + **JSON** + Exception Handling in Python

Recap

- **Minor Project #1**
 - Revised briefing
- **JSON**

Today's Agenda

- Minor Project #1: Expectations & Evaluation
- JSON
- Exception Handling
 - Errors & Exceptions
 - Built-in exceptions
 - User defined exceptions
 - Try – Except block
 - Try – Except – Else Block
 - Try – Except – Else – Finally Block



MDMS: Expectations



1234567.xlsx

Your daily activity capture in the specified format



1234567.py^{*}

Your python functions library through which parameters are calculated



1234567.docx

Your report in the specified format with each parameter & observations

** Python file is optional for Minor Project #1 but will be required for further evaluations*



CAP776 — Minor Project #1 Agentic Evaluator

My Data, My Story • No viva • independent evidence verification • 9 Agents on the GO... © Lovi Raj Gupta

▼ What this agentic evaluator does

NINE Independent agents read the XLSX, validate records, recalculate all prescribed parameters, compare them with the submitted report, check the three prescribed relationships, review evidence in the narrative, and optionally inspect the student Python library.

Student XLSX



Drag and drop file here

Limit 200MB per file • XLSX

Browse files

Submitted Report (.docx)



Drag and drop file here

Limit 200MB per file • DOCX

Browse files

Student Python Library (.py) — strongly recommended



Drag and drop file here

Limit 200MB per file • PY

Browse files



L P U

NAAC
GRADE **A++**

Student XLSX



Drag and drop file here

Limit 200MB per file • XLSX

Browse files



1119.xlsx 23.8KB



Submitted Report (.docx)



Drag and drop file here

Limit 200MB per file • DOCX

Browse files



1119_Report.docx 30.6KB



Student Python Library (.py) — strongly recommended



Drag and drop file here

Limit 200MB per file • PY

Browse files



1119.py 5.3KB



Evaluate



Evaluation completed.

Raw

48.0/100

Scaled

7.2/15

Valid days

40

DCI

100.0%

Parameter Truthfulness

Index	Reported	Recalculated	Status
PAI	331.54	331.54	MATCH
TPI	106.12	106.12	MATCH
AAI	285.75	285.75	MATCH
PhAI	48.25	48.25	MATCH
SRI	449.38	449.38	MATCH
ABI	399.88	399.88	MATCH
TUI	1040.12	1040.12	MATCH
EI	3.4	3.4	MATCH
DCI	100	100	MATCH

CAP776 – Minor Project #1

Agentic Evaluation Report

Student	Student 1119
Registration	1119
Recording period	2026-08-13 to 2026-09-21
Valid days	40
Raw score	48.0/100
Scaled score	7.2/15

1. Parameter Truthfulness

Index	Reported	Recalculated	Status
PAI	331.54	331.54	MATCH
TPI	106.12	106.12	MATCH
AAI	285.75	285.75	MATCH
PhAI	48.25	48.25	MATCH
SRI	449.38	449.38	MATCH
ABI	399.88	399.88	MATCH
TUI	1040.12	1040.12	MATCH
EI	3.40	3.40	MATCH
DCI	100.00	100.00	MATCH



1

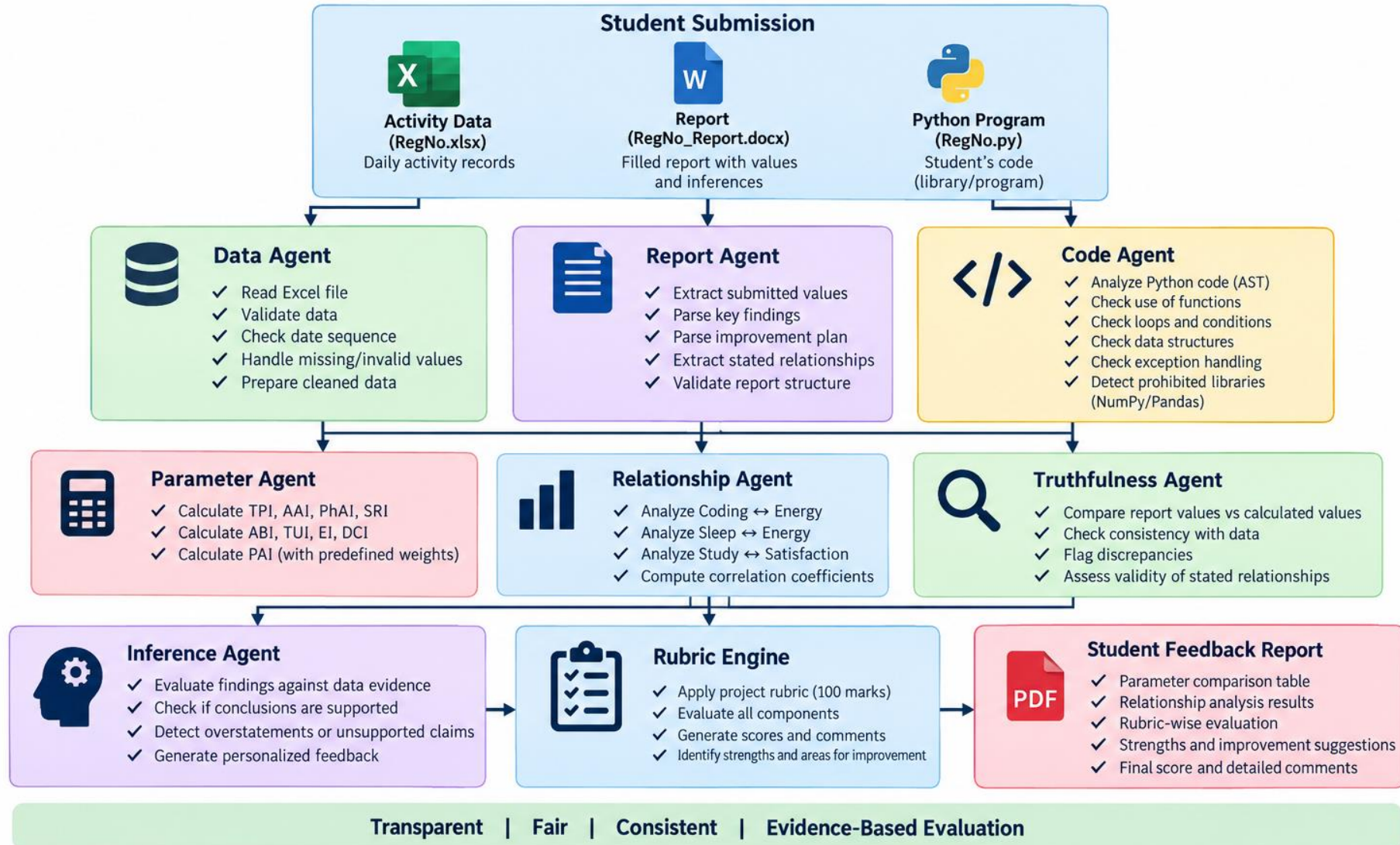


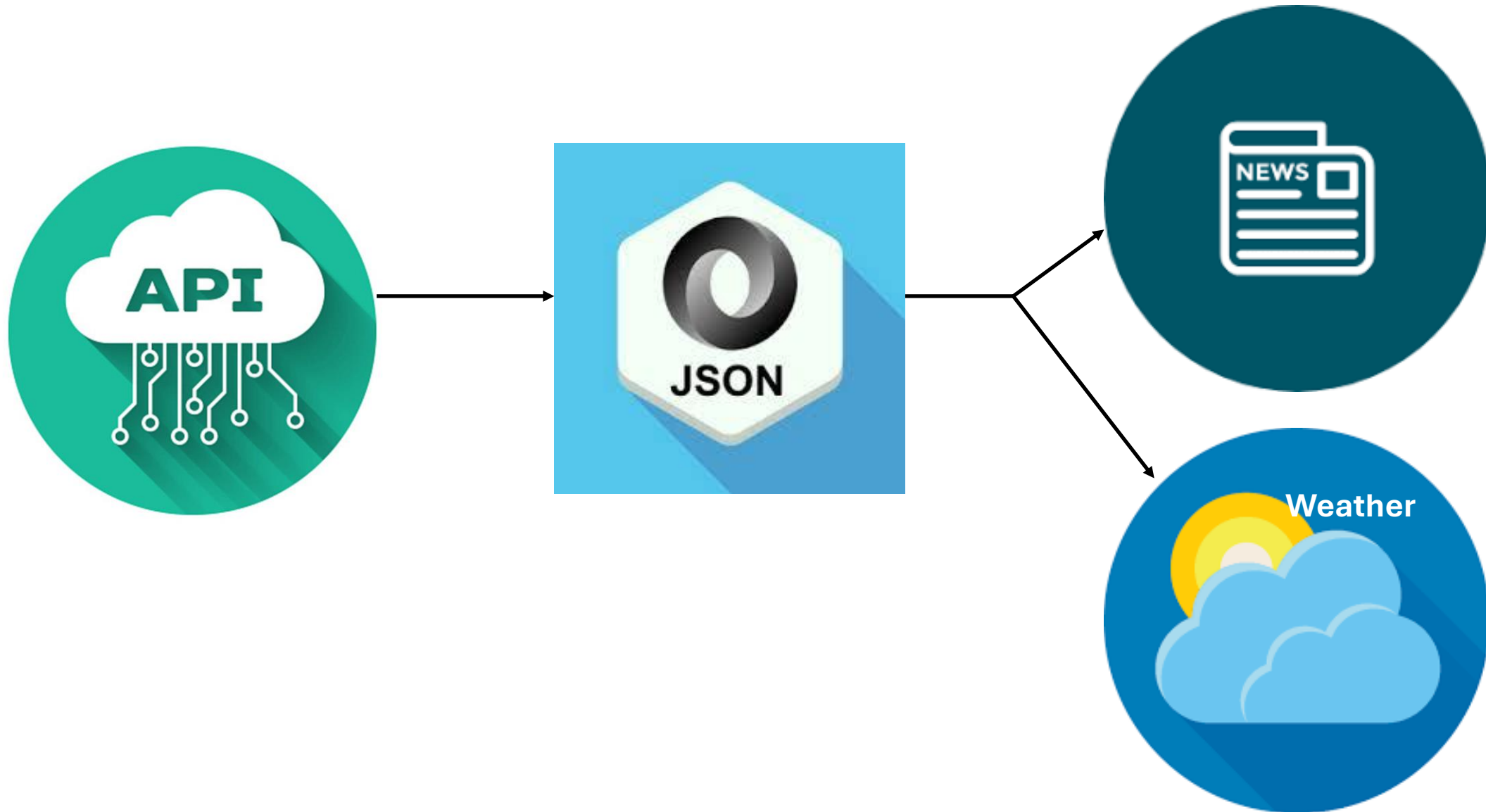
2



CAP776 – Minor Project #1

Agentic Evaluation Architecture





JSON

A JSON (**JavaScript Object Notation**) file is a lightweight, text-based format for storing and exchanging data. It is commonly used for transmitting data between servers and web applications.

JSON Data Structure:

Objects: In JSON, an object is a collection of key-value pairs, where the key is a string and the value can be a string, number, array, object, or boolean.

Arrays: An array is an ordered list of values.

Values: Values can be strings, numbers, arrays, objects, or booleans (true or false).

JSON Object (similar to Python dictionary):

```
{  
  "name": "Kumar",  
  "age": 30,  
  "email": "Kumar@example.com",  
  "languages": ["Java", "Python"]  
}
```

JSON Array (list of objects):

```
[  
  {  
    "name": "Kumar",  
    "email": "Kumar@example.com"  
  },  
  {  
    "name": "Ravi",  
    "email": "Ravi@example.com"  
  }  
]
```

Python Code to Read a JSON File

Step1: import json

Step2: Set Path to your JSON file

Step3: Use file handling method to Open and read the JSON file

Step4: Load JSON data into a Python dictionary

Step5: Accessing the data

- Suppose we have a data.json file in my system

	commutechime.spec	9/1/2024 11:51 AM	SPEC File	1 KB
	data.json	9/16/2024 11:59 PM	JSON Source File	1 KB
	departure_announcement.mp3	8/24/2024 4:29 AM	MP3 File	123 KB

This json file contains:

```
{  
  "name": "Kumar",  
  "age": 30,  
  "email": "Kumar@example.com",  
  "languages": ["Java", "Python"]  
}
```

Code Snippet:

```
import json
# Path to your JSON file
file_path = "data.json"
# Open and read the JSON file
file=open(file_path,"r")
# Load JSON data into a Python dictionary
data = json.load(file)
# Accessing the data
print("Name:", data["name"])
print("Email:", data["email"])
print("Age:", data["age"])
print("Languages:", data["languages"])
```

Name: Kumar

Email: Kumar@example.com

Age: 30

Languages: ['Java', 'Python']

Retrieve data from an API

When you retrieve data from an API using requests, the data is usually in the form of a JSON response, which you can handle using `response.json()`.

`json.load()`: This is used to read JSON data from a file object. It parses the JSON data and converts it into a Python dictionary or list.

```
import json

file_path = "data.json"
file=open(file_path,"r")
data = json.load(file)
print(data)
```

```
{'name': 'Kumar', 'age': 30, 'email': 'Kumar@example.com', 'languages': ['Java', 'Python']}
```

Retrieve data from an API

response.json(): When you use the requests library to make an HTTP request to an API, you get a response object. You can use response.json() to parse the JSON data from the API response directly into a Python dictionary or list.

```
import requests
response = requests.get('https://newsapi.org/v2/everything')
data = response.json()
print(data)
```

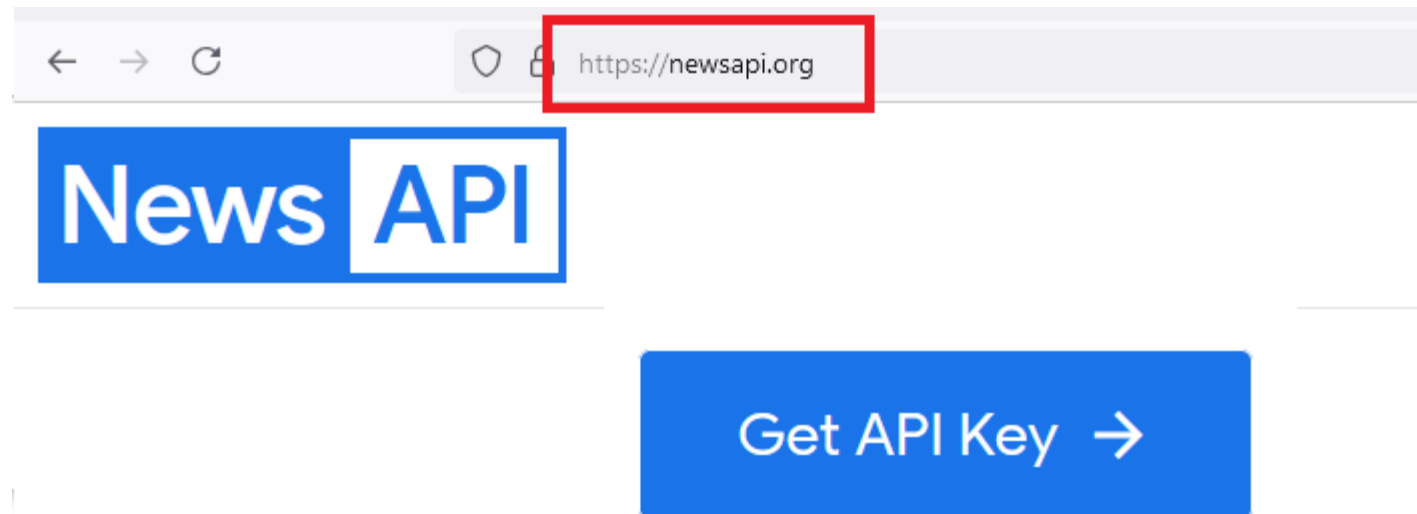
```
{'status': 'ok', 'totalResults': 34795, 'articles': [{ 'source': { 'id': None, 'name': 'BBC News', 'title': 'LGBT Indians demand end to 'discriminatory' ban on blood donation', 'description': 'The UK has lifted bans on blood donations by LGBT people, but India hasn't yet.', 'url': 'https://www.bbc.com/news/health-64422424', 'urlToImage': 'https://ichef.bbci.co.uk/news/1024/branded_news/56ff-11ef-aa88-e1e8b52e1b59.jpg', 'publishedAt': '2024-08-22T01:42:24Z', 'content': 'Indian transgender and gay and bisexual men to donate blood\r\n\r\nIn 2018, India's top court legalised transgender people to donate blood - but the country still doesn't allow transgender people to donate blood... [+6442 chars]'}, { 'source': { 'id': None, 'name': 'BBC News', 'title': 'Diplomatic tightrope for Modi as he visits Kyiv', 'description': 'Modi is visiting Ukraine just weeks after his Russia trip. Is it a balancing act or a thing more?', 'url': 'https://www.bbc.com/news/world-india-64422424', 'urlToImage': 'https://ichef.bbci.co.uk/news/1024/branded_news/af61/live/10d18900-605c-11ef-b3ab-d92ed8dd7357.jpg', 'publishedAt': '2024-08-22T01:42:24Z', 'content': 'Modi is visiting Ukraine just weeks after his Russia trip. Is it a balancing act or a thing more?' } } ] }
```

Sandbox#2:Use News API

Step1: !pip install **requests**

Step2: Create account in <https://newsapi.org/>

And generate API Key





Step3: Use your API Key in url :

```
https://newsapi.org/v2/everything?q=tesla&from=2024-08-16&sortBy=publishedAt&apiKey=API_KEY
```



Code Snippet:

```
import requests

# API endpoint and parameters
url = 'https://newsapi.org/v2/everything'
params = {
    'q': 'india',
    'from': '2024-08-16',
    'sortBy': 'top',
    'apiKey': 'daXXXXXXXXXXXXX988d19353',
    'pageSize': 10 # Limit to top 10 articles
}

# Make the API request
response = requests.get(url, params=params)

data = response.json()
```

Code Snippet:

```
# Extract the articles
articles = data.get('articles', [])

# Counter variable to track article number
article_number = 1

# Print each article
for article in articles:
    print(f"Article {article_number}:")
    print(f"Title: {article['title']}")
    print(f>Description: {article['description']}")
    print(f"URL: {article['url']}\n")

# Increment the counter
article_number += 1
```

Article 1:

Title: LGBT Indians demand end to 'discriminatory' ba
Description: Many countries have lifted bans on blood
URL: <https://www.bbc.com/news/articles/clywepq2eq2o>

Article 2:

Title: Diplomatic tightrope for Modi as he visits Kyi
Description: Modi is visiting Ukraine just weeks after
hing more?
URL: <https://www.bbc.com/news/articles/cx2e9zrywq9o>

Article 3:

Title: Accusations that Amazon Mistreats Workers in I
Description: Just like in the U.S., Amazon is a big e
n's fulfillment centers in India have been the subject
stand what those conditions are like, we hear from...

Think the situation like:



You leave the house
and it suddenly starts
raining, but you forgot
to bring your
umbrella.



Another situation like:

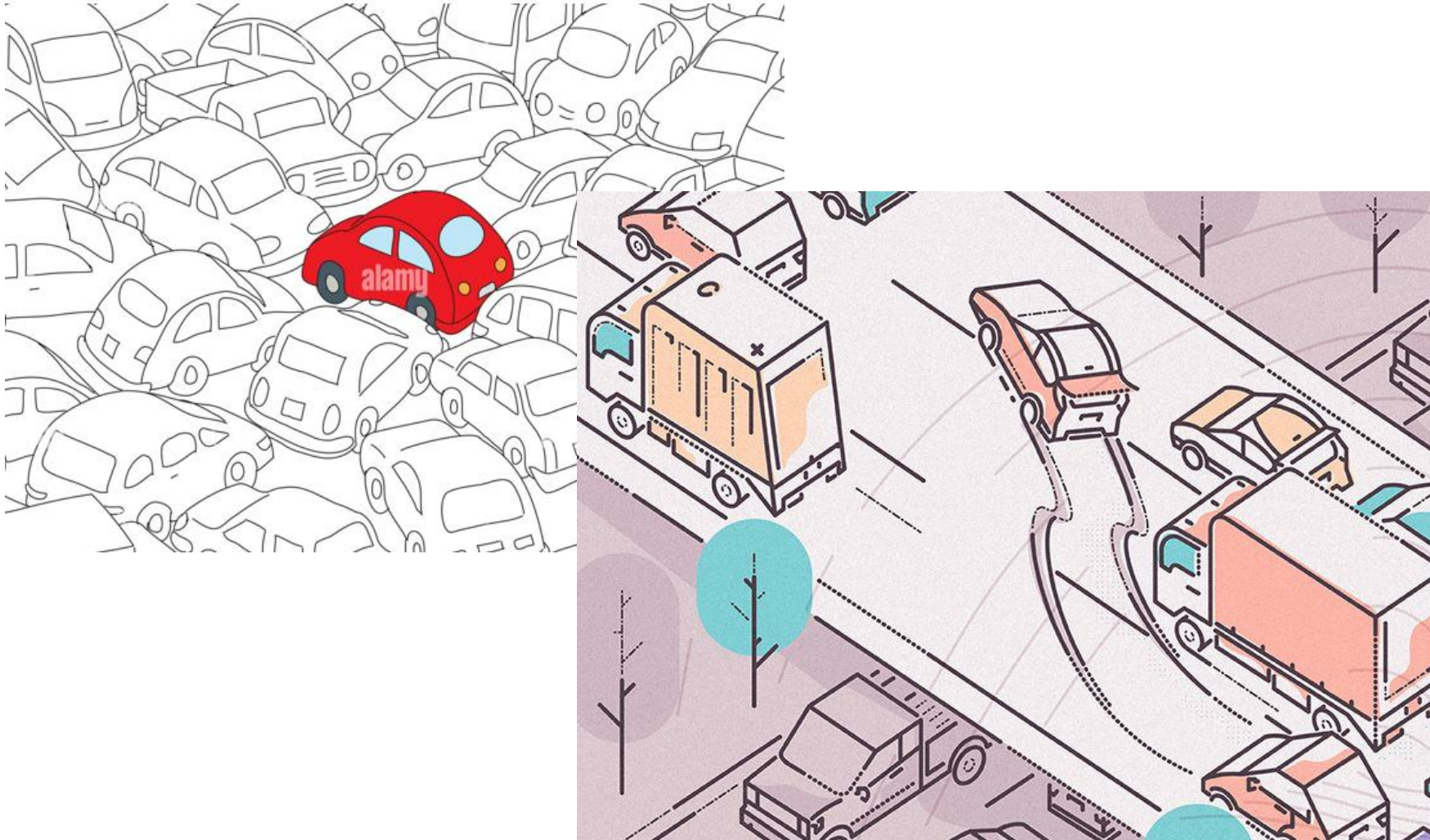


You set an alarm to wake up early, but for some reason, the alarm doesn't ring



**Exceptions are nothing, it is just like
a unexpected situation**

Road state:



Road state:



Code state: Errors

- **Syntax Errors: Would cease the program**

- **Wrong syntax**

- Misspelled keyword
- Unbalanced parentheses
- Missing colon...

```
# code to calculate exponent value of an input
num = int(input("Enter a number: "))
exp = int(input("Enter the exponent: "))
print(num**exp)
```

```
# code to calculate exponent value of an input
num = int(input("Enter a number: "))
exp = int(input("Enter the exponent: "))
print(num**exp)
```

File "<ipython-input-3-ac60fc45b56f>", line 3

```
exp = int(input("Enter the exponent: "))
```

^

SyntaxError: '(' was never closed

Code state: Errors

- **Exceptions:** Would disrupt the flow

```
# Simple division program without exception handling
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

result = num1 / num2
print(f"The result of division is: {result}")
```

Enter the first number: 10

Enter the second number: 0

ZeroDivisionError

Traceback (most recent call last):

Cell In[4], line 5

```
2 num1 = int(input("Enter the first number: "))
```

```
3 num2 = int(input("Enter the second number: "))
```

```
----> 5 result = num1 / num2
```

```
6 print(f"The result of division is: {result}")
```

ZeroDivisionError: division by zero

try and except block

try:

#block of code

except Exception1:

#block of code

except Exception2:

#block of code

#other code

try block: The code inside the try block is executed, and if an exception occurs, control is transferred to the appropriate except block.

except block: Catches the exception and handles it. You can specify the exception type or catch all exceptions using a generic except block.

Handling exceptions

```
try:
    num1 = int(input("Enter the first number: "))
    num2 = int(input("Enter the second number: "))

    result = num1 / num2
    print(f"The result of division is: {result}")
except ValueError:
    print("Invalid input! Please enter valid numbers.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

```
Enter the first number: 10
Enter the second number: 0
Cannot divide by zero!
```

Different types of exceptions

- Exception can be categories in two different types:
 - ✓ Built-in exception and
 - ✓ user define exception

Built-in Python exceptions

In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program.



<https://docs.python.org/3/library/exceptions.html#exception-hierarchy>

Exception hierarchy

The class hierarchy for built-in exceptions is:

```
BaseException
├── BaseExceptionGroup
├── GeneratorExit
├── KeyboardInterrupt
├── SystemExit
├── Exception
│   ├── ArithmeticError
│   │   ├── FloatingPointError
│   │   ├── OverflowError
│   │   └── ZeroDivisionError
│   ├── AssertionError
│   ├── AttributeError
│   ├── BufferError
│   ├── EOFError
│   ├── ExceptionGroup [BaseExceptionGroup]
│   ├── ImportError
│   │   └── ModuleNotFoundError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── MemoryError
│   ├── NameError
│   │   └── UnboundLocalError
```

Built-in Python exceptions

- **TypeError:** Raised when an operation or function is applied to an object of an inappropriate type.

```
a = '2' + 2 # TypeError: cannot concatenate 'str' and 'int'
```

- **ValueError:** Raised when a function receives an argument of the correct type but an inappropriate value.

```
int("abc") # ValueError: invalid literal for int()
```

Built-in Python exceptions

- **IndexError:** Raised when trying to access an element from a list or other sequence using an invalid index.

```
my_list = [1, 2, 3]  
print(my_list[5]) # IndexError: list index out of range
```

- **KeyError:** Raised when trying to access a dictionary with a key that doesn't exist.

```
my_dict = {'a': 1}  
print(my_dict['b']) # KeyError: 'b'
```

Catching Multiple Exceptions:

- You can handle multiple exceptions by chaining except blocks
- Or even catch multiple exceptions in one block

```
try:
    # Code that may raise exceptions
    ...
except ExceptionType1:
    # Handle ExceptionType1
    ...
except ExceptionType2:
    # Handle ExceptionType2
    ...
except ExceptionType3:
    # Handle ExceptionType3
    ...
```

```
try:
    # Code that may raise exceptions
    pass
except (ExceptionType1, ExceptionType2, ExceptionType3) as e:
    # Handle multiple exceptions here
    print(f"Error: {e}")
```

Generic Exception Handling:

- If you want to catch any exception, you can use a generic except block.

```
try:  
    # some code  
except Exception as e:  
    print(f"An error occurred: {e}")
```

else statement with the try-except

try:

#block of code

except Exception1:

#block of code

else:

#this code executes if no
except block is executed

use the else statement with the try-except statement

try:

a=int(input("enter a"))

b=int(input("enter b"))

c=a/b

print(c)

except:

print("Invalid input")

else:

print("Thank you")

enter a6

enter b3

2.0

Thank you

try...finally block

try:

block of code
this may throw an exception

finally:

block of code
this will always be executed

example of finally block

try:

#this will throw an exception if the file doesn't exist.

```
fileptr = open("file.txt","r")
```

except IOError:

```
print("File not found")
```

finally:

```
fileptr.close()
```

```
print("finally block")
```

File not found

finally block

raise keyword

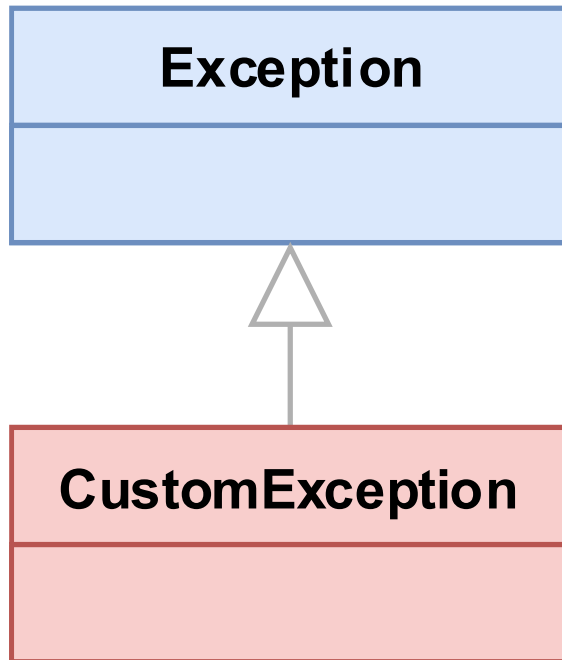
In Python, you can explicitly throw (or raise) an exception using the **raise** keyword. This is especially useful when you want to create user define exception.

Syntax

```
raise Exception_class,<value>
```

Custom exception or User define Exception

Users can define custom exceptions by creating a new class. This exception class has to be derived from the built-in Exception class.



```
class CustomException(Exception):  
    def __init__(self,msg='Exception occurs'):  
        self.msg = msg  
        super().__init__(self.msg)  
  
try:  
    raise CustomException()  
except CustomException as ex:  
    print(ex)
```

Exception occurs

Sandbox#1

Imagine you're processing a list of orders for a restaurant. Each order has the customer's name and the total amount they need to pay. You want to calculate the total bill for all orders, but need to handle potential errors, such as:

- ☐ `ValueError`: If the order amount contains an invalid value (e.g., text instead of a number).
- ☐ `FileNotFoundError`: If the file containing the orders is missing.



Solution: Step#1: Define a function which process order

```
def process_orders(orders):  
    total = 0  
    for order in orders:  
        try:  
            item = order["item"]  
            amount = int(order["amount"])  
            # May raise ValueError if amount is not a valid integer  
  
            if amount <= 0:  
                raise ValueError("Order amount must be positive.")  
  
            # Add the valid amount to the total  
            total += amount  
  
        except KeyError as ke:  
            print(f"Error: Missing key in order - {ke}")  
  
        except ValueError as ve:  
            print(f"Error: Invalid amount in order for {item}. {ve}")  
  
    return total
```

Step#2: Take orders list which contain wrong value

```
# List of orders with electronics items
orders = [
    {"name": "Manu", "item": "Smartphone", "amount": "100"},
    {"name": "Raju", "item": "Laptop", "amount": "abc"},
    {"name": "Kunal", "item": "Headphones", "amount": "30"}
]

try:
    total_amount = process_orders(orders)
except Exception as e:
    print(f"An unexpected error occurred: {e}")
else:
    print(f"Total amount of all valid orders: ${total_amount}")
```

Output:

```
Error: Invalid amount in order for Laptop. invalid literal for int() with base 10: 'abc'
Total amount of all valid orders: $130
```

Sandbox#2 : Breaking Speed Limit

A driver must adhere to a speed limit. If the speed exceeds the limit, a custom exception `SpeedLimitExceededError` is raised.



Solution: Step#1: define exception class

```
class SpeedLimitExceededError(Exception):  
    def __init__(self, speed, message="Your speed exceeds the limit"):  
        self.speed=speed  
        self.message = message  
        super().__init__(self.message)
```

Step#2: define function to check speed limit

```
def checkSpeed(speed):  
    if speed>80:  
        raise SpeedLimitExceededError(speed)  
    else:  
        print(f"Your speed is within the limit. Current speed: {speed} km/h")
```

Step#3: handle the exception using try and except

```
try:
    speed = int(input("Enter Speed "))
    checkSpeed(speed)
except SpeedLimitExceededError as e:
    print(f"Error: {e.message}. Your speed is: {e.speed} Km/h")
```

Enter Speed 90

Error: Your speed exceeds the limit. Your speed is: 90 Km/h



Up next...

- OOP